

Assignment Number: 4 (HPC)**Name:** Hadawale Aniket Ashok**Roll No.:** 24**Assignment Title:** Write a CUDA Program for : 1. Addition of two large vectors 2. Matrix Multiplication using CUDA C**CODE (Vector Addition)**

```
// =====  
// Assignment 4: CUDA Programs  
// Part 1 – Addition of two large vectors  
// Part 2 – Matrix Multiplication  
// =====
```

```
#include <stdio.h>  
#include <cuda_runtime.h>  
#include <time.h>
```

```
#define N 10000000 // 10 million elements
```

```
// ——— CUDA Kernel ———  
__global__ void vectorAdd(const float* A, const float* B, float* C, int n) {  
    int idx = blockIdx.x * blockDim.x + threadIdx.x;    if (idx < n) C[idx] =  
    A[idx] + B[idx];  
}
```

```
// ——— CPU Reference ———  
void vectorAdd_CPU(const float* A, const float* B, float* C, int n) {  
    for (int i = 0; i < n; i++) C[i] = A[i] + B[i];  
}
```

```
int main() {    size_t bytes = N *  
    sizeof(float);
```

```
    // Host allocations    float *h_A =  
    (float*)malloc(bytes);    float *h_B =  
    (float*)malloc(bytes);    float *h_C =  
    (float*)malloc(bytes); // GPU result    float  
    *h_C_CPU = (float*)malloc(bytes); // CPU result
```

```
    for (int i = 0; i < N; i++) { h_A[i] = i * 0.5f; h_B[i] = i * 0.3f; }
```

```

// Device allocations
float *d_A, *d_B, *d_C;
cudaMalloc(&d_A, bytes);
cudaMalloc(&d_B, bytes);  cudaMalloc(&d_C,
bytes);

// — CPU timing —————
clock_t cpu_start = clock();  vectorAdd_CPU(h_A, h_B, h_C_CPU, N);  double cpu_ms =
(double)(clock() - cpu_start) / CLOCKS_PER_SEC * 1000.0;  printf("CPU Vector Add time:
%.2f ms\n", cpu_ms);

// — GPU timing —————
cudaEvent_t start, stop;  cudaEventCreate(&start); cudaEventCreate(&stop);

  cudaMemcpy(d_A, h_A, bytes, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, bytes, cudaMemcpyHostToDevice);

  int threadsPerBlock = 256;  int blocksPerGrid  = (N +
threadsPerBlock - 1) / threadsPerBlock;

  cudaEventRecord(start);  vectorAdd<<<blocksPerGrid,
threadsPerBlock>>>(d_A, d_B, d_C, N);  cudaEventRecord(stop);
cudaEventSynchronize(stop);

  float gpu_ms = 0;  cudaEventElapsedTime(&gpu_ms, start, stop);
printf("GPU Vector Add time: %.2f ms (Blocks=%d, Threads=%d)\n",
gpu_ms, blocksPerGrid, threadsPerBlock);  printf("Speedup: %.2fx\n",
cpu_ms / gpu_ms);

  cudaMemcpy(h_C, d_C, bytes, cudaMemcpyDeviceToHost);

// Verify  int errors = 0;  for (int i = 0; i < N; i++)  if (fabs(h_C[i] -
h_C_CPU[i]) > 1e-4) errors++;  printf("Verification: %s (%d errors)\n", errors == 0 ?

```

```
"PASSED" : "FAILED", errors); // Cleanup  cudaFree(d_A); cudaFree(d_B);
cudaFree(d_C);  free(h_A); free(h_B); free(h_C); free(h_C_CPU);
cudaEventDestroy(start); cudaEventDestroy(stop);  return 0;
}
```

```
// Compile: nvcc -O2 -o vector_add vector_add.cu && ./vector_add
```

CODE (Matrix Multiplication)

```
// -----
// PART 2: Matrix Multiplication
// FILE: matrix_mul.cu
// -----
```

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <time.h>
#include <cuda_runtime.h>
```

```
#define N 512 // Matrix size N x N
#define TILE 16 // Tile size for shared memory optimisation
```

```
// — Naive CUDA Kernel —
__global__ void matMul_Naive(const float* A, const float* B, float* C, int n) {
    int row = blockIdx.y * blockDim.y + threadIdx.y;  int col = blockIdx.x *
    blockDim.x + threadIdx.x;  if (row < n && col < n) {      float sum = 0.0f;
    for (int k = 0; k < n; k++)
        sum += A[row * n + k] * B[k * n + col];
        C[row * n + col] = sum;
    }
}
```

// — Tiled Shared-Memory CUDA Kernel —————

```
__global__ void matMul_Tiled(const float* A, const float* B, float* C, int n) {
    __shared__ float sA[TILE][TILE];
    __shared__ float sB[TILE][TILE];

    int row = blockIdx.y * TILE + threadIdx.y;
    int col = blockIdx.x * TILE + threadIdx.x;
    float sum = 0.0f;

    for (int t = 0; t < (n + TILE - 1) / TILE; t++) {
        sA[threadIdx.y][threadIdx.x] =
            (row < n && t*TILE + threadIdx.x < n)
            ? A[row * n + t*TILE + threadIdx.x] : 0.0f;
        sB[threadIdx.y][threadIdx.x] =
            (col < n && t*TILE + threadIdx.y < n)
            ? B[(t*TILE + threadIdx.y) * n + col] : 0.0f;
        __syncthreads();

        for (int k = 0; k < TILE; k++) sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
        __syncthreads();
    }
    if (row < n && col < n) C[row * n + col] = sum;
}
```

// — CPU Matrix Multiply —————

```
void matMul_CPU(const float* A, const float* B, float* C, int n) {
    for (int i = 0; i < n; i++)        for (int j = 0; j < n; j++) {
        float s = 0;
        for (int k = 0; k < n; k++) s += A[i*n+k] * B[k*n+j];
        C[i*n+j] = s;
    }
}
```

```

int main() {    int size = N * N;
size_t bytes = size * sizeof(float);

    float *h_A = (float*)malloc(bytes);
float *h_B = (float*)malloc(bytes);    float
*h_C_naive = (float*)malloc(bytes);    float
*h_C_tiled = (float*)malloc(bytes); float
*h_C_cpu  = (float*)malloc(bytes);

    srand(42);    for (int i = 0; i < size; i++)
{        h_A[i] =
(float)rand()/RAND_MAX;        h_B[i] =
(float)rand()/RAND_MAX;
    }

    // — CPU —————
clock_t c1 = clock();    matMul_CPU(h_A, h_B, h_C_cpu, N);    double cpu_ms = (double)(clock()-
c1)/CLOCKS_PER_SEC*1000.0;
printf("\n=====\\n");    printf(" Matrix
Multiplication (%dx%d)\\n", N, N);
    printf("=====\\n");
    printf("CPU time:        %.2f ms\\n", cpu_ms);

    // Device allocation    float *d_A, *d_B, *d_C;
cudaMalloc(&d_A, bytes);    cudaMalloc(&d_B, bytes);
cudaMalloc(&d_C, bytes);    cudaMemcpy(d_A, h_A, bytes,
cudaMemcpyHostToDevice);    cudaMemcpy(d_B, h_B, bytes,
cudaMemcpyHostToDevice);

    dim3 block(TILE, TILE);    dim3 grid((N+TILE-
1)/TILE, (N+TILE-1)/TILE);

```

```

    cudaEvent_t start, stop;
cudaEventCreate(&start); cudaEventCreate(&stop);
float gpu_ms;

// — Naive GPU —————
cudaEventRecord(start);  matMul_Naive<<<grid, block>>>(d_A, d_B, d_C, N);
cudaEventRecord(stop); cudaEventSynchronize(stop);  cudaEventElapsedTime(&gpu_ms, start,
stop);  cudaMemcpy(h_C_naive, d_C, bytes, cudaMemcpyDeviceToHost);  printf("GPU
Naive time:  %.2f ms Speedup: %.2fx\n", gpu_ms, cpu_ms/gpu_ms);

// — Tiled GPU —————
cudaEventRecord(start);  matMul_Tiled<<<grid, block>>>(d_A, d_B, d_C, N);
cudaEventRecord(stop); cudaEventSynchronize(stop);  cudaEventElapsedTime(&gpu_ms, start,
stop);  cudaMemcpy(h_C_tiled, d_C, bytes, cudaMemcpyDeviceToHost);
printf("GPU Tiled time:  %.2f ms Speedup: %.2fx\n", gpu_ms, cpu_ms/gpu_ms);

// Verify (compare naive and tiled vs CPU)  int err_n
= 0, err_t = 0;  for (int i = 0; i < size; i++) {      if
(fabs(h_C_naive[i] - h_C_cpu[i]) > 1e-2) err_n++;      if
(fabs(h_C_tiled[i] - h_C_cpu[i]) > 1e-2) err_t++;
    }
    printf("Naive Verify: %s Tiled Verify: %s\n",
err_n==0?"PASS":"FAIL", err_t==0?"PASS":"FAIL");

    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);  free(h_A);
free(h_B); free(h_C_naive); free(h_C_tiled); free(h_C_cpu);
cudaEventDestroy(start); cudaEventDestroy(stop);  return 0;
}

// — Compile & Run —————
// nvcc -O2 -o matrix_mul assignment4_cuda.cu && ./matrix_mul

```

OUTPUT

```
In [ ]: # Assignment Number: 4 (a)
# Name: Hadawale Aniket Ashok
# Roll No.: 24
# Assignment Title: Write a CUDA Program for :
# 1. Addition of two Large vectors
# 2. Matrix Multiplication using CUDA C
```

```
In [ ]: !nvidia-smi
```

```
Wed Apr 15 16:27:16 2026
+-----+
| NVIDIA-SMI 580.82.07              Driver Version: 580.82.07      CUDA Version: 13.0     |
+-----+-----+
| GPU  Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap|   Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
| 0 Tesla T4               Off          | 00000000:00:04:0 Off |                    0 |
| N/A   51C    P8             14W /  70W |  0MiB / 15360MiB |      0%    Default  |
+-----+-----+

+-----+
| Processes:                         |
| GPU   GI    CI          PID    Type   Process name          GPU Memory |
| ID    ID    ID              |              | Usage       |
+-----+-----+
| No running processes found.      |
+-----+
```

```
In [ ]: !nvcc --version
```

```
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2025 NVIDIA Corporation
Built on Fri,Feb,21,20:23:50_PST_2025
Cuda compilation tools, release 12.8, V12.8.93
Build cuda_12.8.r12.8/compiler.35583870_0
```

1. Vector Addition

```
In [ ]: from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving assignment4_vector_add.cu to assignment4_vector_add.cu

```
In [ ]: !nvcc -arch=sm_75 assignment4_vector_add.cu -o vector
```

```
In [ ]: !./vector
```

```
CPU Vector Add time: 48.81 ms
GPU Vector Add time: 0.52 ms (Blocks=39063, Threads=256)
Speedup: 93.31x
Verification: PASSED (0 errors)
```

2. Matrix Multiplication

```
In [ ]: from google.colab import files
uploaded = files.upload()
```

Choose Files No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving assignment4_matrix_mul.cu to assignment4_matrix_mul.cu

```
In [ ]: !nvcc -arch=sm_75 assignment4_matrix_mul.cu -o matrix
```

```
In [ ]: !./matrix
```

```
=====
Matrix Multiplication (512x512)
=====
CPU time:      775.45 ms
GPU Naive time: 0.74 ms Speedup: 1043.57x
GPU Tiled time: 0.47 ms Speedup: 1652.31x
Naive Verify: PASS   Tiled Verify: PASS
```